

От нуля до MVP: задачи Go-разработчика

Практический маршрут для сервера ИИ-помощника: Go получает факты ЕКТ, строит безопасный ответ на вопрос клиента, подбирает проверенную замену и управляет подтверждением корзины. В материалах нет подтверждённого метода корзины ЕКТ.

Что Go должен отдавать фронту

Главный результат — ответ `POST /api/chat`: понятный текст, структурированные карточки товара и, при запросе покупки, отдельное предложение действия. Для этого Go получает и проверяет данные ЕКТ. Поиск и карточка также доступны React для отображения и ручной навигации. Ключ LLM хранится только на сервере.

Обязанность Go	Минимальный результат
ИИ-помощник	Разобрать вопрос; ответить по проверенным полям; вернуть карточки и предложение действия
ЕКТ-клиент + данные	Получить список и detail; собрать единый Product с data_issues
Поиск/аналоги	Найти ID/артикул и проверить кандидатов на замену
Корзина	Два шага: предложение → явное подтверждение → новая проверка → действие

При отсутствии настоящего API корзины сервер возвращает `mode: "demo"`. Демо-корзина полезна для показа логики, но не означает выполненную интеграцию с `ekt.kz`.

Критерий готовности бэка

- Вопрос «Есть ли 515291?» проходит путь чат → данные ЕКТ → проверенный ответ с карточкой.
- Все согласованные маршруты отвечают структурированным JSON и правильными HTTP-кодами.
- Остаток, цена и характеристики взяты из ЕКТ detail, а не придуманы LLM.
- Повторный confirm и запрос сверх остатка не создают лишних позиций.

Этап 0. Создать и запустить Go-сервис

Проверить go version. Для приведённого ниже синтаксиса маршрутов с {id} нужна Go 1.22+; на более старой версии использовать совместимый роутер или ручной разбор пути. Создать отдельную папку backend и модуль Go:

```
mkdir backend
cd backend
go mod init ekt-navigator/backend
```

В main.go создать http.NewServeMux, порт :8080 и маршрут проверки GET /api/health, который возвращает {"status":"ok"}. Затем запустить go run . и открыть http://localhost:8080/api/health.

Настройки держать вне кода: EKT_BASE_URL, PORT, ключ LLM, CART_MODE=demo|live. Не передавать ключи в React. В README записать команду запуска и требуемые переменные. Для реальной корзины заранее выяснить у партнёра авторизацию и метод интеграции.

Результат этапа: сервер стартует отдельно от React и отдаёт JSON health. Если Go-сервис уже есть в репозитории, добавить маршруты в него, не создавая второй конкурирующий сервер.

Этап 1. Согласовать контракт с фронтом

Маршрут вашего Go API	Ответ, который ждёт React
GET /api/search?q=...	{items: [ProductSummary]}
GET /api/products/{id}	Один нормализованный Product
GET /api/products/{id}/alternatives	{source_product_id, items: [{product, reason, differences}]}
POST /api/chat	{reply, products: [], pending_action: null {product_id, quantity, confirmation_token}}
POST /api/cart/confirm	{status, mode, cart_url?}; 409 если цена/остаток изменились

Ошибки отдавать в виде {"error":"понятное сообщение","code":"..."} с HTTP 400 (неверный запрос), 404 (товар не найден), 409 (изменились условия), 502/503 (ЕКТ недоступен). Сессия и confirmation_token должны быть привязаны друг к другу.

Этап 2. Сделать надёжный ЕКТ-клиент

- Реализовать запросы к подтверждённым методам ЕКТ: GET /api/products и GET /api/products/detail?id={id}. Переданные страницы списка содержат id, name, article, price, image, url, url_api_detail, offers; подробная карточка содержит quantity, stores, description, properties.
- Использовать один http.Client с разумным Timeout; передавать контекст входящего запроса, проверять HTTP-статус и ограничения размера ответа, аккуратно разбирать JSON. При сбое возвращать явную ошибку, а не старый остаток.
- Не предполагать, что у ЕКТ есть текстовый поиск или подтверждённый cart API: таких методов в предоставленных примерах нет.

Первые ручные проверки

Запрос	Что зафиксировать
Список товаров	Поля и пагинацию; count=20 в образце означает число элементов текущего ответа, а не размер всего каталога.
Detail 515291	ID, article="200300285_", price=64920 и quantity=23 в переданном снимке.
Склады	Сумма положительных складов из образца = 23; некоторые складские статусы могут не означать продажу.
properties	RECOMM = связанные ID, не автоматические аналоги; ток в разных полях противоречив.

Результат этапа: Go умеет стабильно получать и разбирать одну реальную карточку ЕКТ. При недоступности ЕКТ понятно сообщает ошибку.

Этап 3. Нормализовать Product

Следовать DATA_CONTRACT.md: сохранить ID, артикул как строку с завершающим «_», цену, quantity, склады, бренд, свойства, URL, related_product_ids, certificate_urls и data_issues. Список без quantity не доказывает нулевой остаток: использовать null.

- У 515291 name/description дают 160 A, properties.NOMINALNYY_TOK - 250 A. Вывести characteristics.current = null и добавить data_issues; не выбирать одно значение молча.
- Проверить сумму складов и пометить расхождения; склады с названием «Брак» не объявлять доступными к продаже без правила партнёра.

Результат этапа: GET /api/products/515291 возвращает единый Product с конфликтом тока и quantity=23 для переданного образца; свежие значения могут измениться.

Этап 4. ИИ-помощник: вопрос → проверенный ответ

- Согласовать у партнёра тексты оплаты, доставки и минимальной партии; хранить источник/дату правила. Без этих текстов отвечать «условия требуют уточнения». Сертификаты выдавать только по реальной ссылке.
- POST /api/chat принимает message и selected_product_id. Первый полный сценарий: «Есть ли 515291?» → определить ID → запросить EKT detail → нормализовать Product → вернуть reply и products: [Product].
- Использовать LLM для понимания свободного вопроса и формулировки ответа. Передавать ей только проверенные поля; цену, остаток, сертификат и совместимость сервер берёт из данных, а не из текста модели. Проверять структуру ответа перед отправкой React.
- Проверить вопросы про характеристики и условия покупки. Если EKT или LLM недоступны, честно сообщить об ошибке; не выдавать заранее заготовленный текст за актуальное наличие.
- Если пользователь просит «Добавь 2», вернуть отдельный pending_action. Текст reply может объяснить предложение, но не является сигналом изменить корзину.

Результат этапа: ассистент отвечает на живой вопрос по товару и прикладывает карточку. Он показывает неопределённость и не меняет корзину из текста сообщения.

Этап 5. Поиск и подбор проверенного аналога

Поиск для вопросов без точного ID

- Собрать подготовленный индекс примерно 5-10 проверенных товаров. Порядок: точный ID → точный внутренний/поставщицкий артикул → название → частичное название. Сохранять ведущие нули и символ «_» в артикулах.
- GET /api/search?q=... возвращает краткие карточки для React; тот же механизм помогает чату понять запрос по названию. Перед ответом о наличии запрашивать detail; не выдавать выборку за полный каталог.

Аналог как продолжение диалога

- Если товар закончился, взять кандидатов из RECOMM и/или проверенной выборки, для каждого получить detail. Сначала фильтр: quantity > 0, подтверждённый тип и необходимые характеристики.
- Сравнивать число полюсов, номинальный ток, напряжение, отключающую способность и назначение. Допустимые замены должен утвердить предметный эксперт; более высокий номинал сам по себе не доказывает совместимость.
- Вернуть reason из реальных совпадений и differences из отличий в POST /api/chat и GET /api/products/{id}/alternatives. При конфликте критичного параметра вернуть пустой список без обещания совместимости.

Результат этапа: вопрос «Что взять вместо отсутствующего товара?» приводит к проверенному варианту или честному сообщению, что вариант не найден.

Этап 6. Безопасное добавление в корзину

Подготовить предложение

Сохранить на сервере {product_id, quantity, price_snapshot, confirmation_token, session_id, expires_at, used=false}. Токен должен быть одноразовым и связанным с текущей сессией. Пока клиент не подтвердил, корзина не меняется.

Подтвердить

- Получить POST /api/cart/confirm с confirmation_token. Проверить токен, срок, сессию, used и количество; повторно вызвать EKT detail. Не использовать остаток из старого списка или текста LLM.
- Если цена/остаток изменились или quantity превышает доступное количество: 409, без добавления, вернуть новые условия для повторного согласия. Если всё верно: пометить токен использованным, добавить ровно один раз и вернуть статус/ссылку.
- Если настоящего метода корзины пока нет, явно вернуть mode: demo и хранить демонстрационную корзину отдельно от ekt.kz. Не обещать реальную ссылку cart_url, пока она не существует.

Результат этапа: «Добавь 2» само по себе ничего не меняет; после подтверждения есть одна операция; повторный confirm не дублирует товар.

Этап 7. Интеграция с React

- Запустить Go на :8080 и React на :5173. React через Vite proxy отправляет /api/... в Go; frontend не должен обращаться на ekt.kz напрямую.
- Проверить JSON на каждой границе: Go → React. В первую очередь соединить вопрос в чате и карточку ответа, затем аналоги и подтверждение корзины. Для опубликованной версии настроить единый origin или reverse proxy для /api.

Сбой	Результат сервера
Неверный ID / количество	400 либо 404, понятное error и без изменения корзины
Изменился остаток/цена	409, новые условия, без добавления
ЕКТ недоступен	502/503, отсутствие обещания наличия
Повторили token	Без повторного добавления; предсказуемый ответ

Этап 8. Проверка и сдача бэка

Тестовый запрос	Ожидаемое поведение
POST /api/chat «Есть ли 515291?»	Ответ с карточкой, свежим остатком или явным сообщением о недоступности ЕКТ.
GET /api/search?q=200300285_	В результате есть ID 515291; артикул не потерял символ «_».
GET /api/products/515291	Сырые поля приведены к контракту; конфликт 160/250 А указан явно.
Товар с quantity=0	Доступен проверенный аналог либо честный пустой список.
POST /api/chat «Добавь 2»	Есть pending_action, а корзина ещё не изменилась.
POST /api/cart/confirm	Сервер повторно проверил detail; действовал один раз.
Запрос выше остатка	Отказ; без добавления и с предложением корректного числа.
Отключить ЕКТ	Понятная ошибка; устаревший кэш не используется для покупки.

Сдать README с запуском, переменными окружения без секретов, таблицей маршрутов, образцами JSON, списком тестовых ID и честным описанием режима корзины. Небольшая проверка через curl/браузер до соединения с React экономит время команды.

Реалистичный порядок на 5 часов

0:00-0:30 - Go-сервис и контракт; 0:30-1:30 - ЕКТ-клиент и Product; 1:30-2:30 - чат: вопрос о конкретном товаре и ответ; 2:30-3:30 - поиск по названию, аналог и предложение корзины; 3:30-4:30 - подтверждение и React-интеграция; 4:30-5:00 - ошибки и демо. Если времени мало, приоритет у законченного вопроса и проверенного ответа.

Завершено: Go обеспечивает работу ИИ-помощника на проверенных данных и не меняет корзину без явного подтверждения. Неизвестные сертификаты, FAQ и аналоги не придумываются.

Источники: приложенное ТЗ ЕКТ MVP; DATA_CONTRACT.md; официальная документация net/http - <https://pkg.go.dev/net/http> и установка Go - <https://go.dev/doc/install>.